

Scientific Python in 2026: The Stack as I'd Run It Today

The modern Python data-stack for MSDS coursework and personal data projects. `uv` for everything Python-as-a-tool. Polars over Pandas. Marimo over Jupyter. Altair as the declarative default; Matplotlib when you need control. Scikit-learn unchanged because nothing has replaced it. PyTorch for deep learning. The whole thing lives in a Dev Container so it's reproducible and disposable.

NOTE. LAST VALIDATED: 2026-05. Python 3.12+, `uv` current, Polars 1.x, Marimo current stable, Altair 5.x, scikit-learn 1.5+, PyTorch 2.x. Bump and re-validate annually per [CLAUDE.md](#) standard #5.

This document covers the **whole stack** as a coherent thing, not "ten tools loosely federated." Each section explains both what the tool is and why it's the current pick — including what it replaced and when you'd reconsider. The cheat sheet at the end is for printing and pinning above the desk.

What you'll have at the end

- A clear mental model of the modern scientific-Python stack and what each tool is for.
- A working Dev Container template for Python 3.12 + `uv` + the scientific stack.
- A repeatable project layout for data projects (raw data, notebooks, scripts, src/, output).
- Opinionated defaults for DataFrames (Polars), notebooks (Marimo), plotting (Altair / Matplotlib / Plotly / Seaborn), classical ML (scikit-learn), and deep learning (PyTorch).
- A reproducibility checklist (lockfile, seeds, dataset handling) that survives 6-month gaps between picking a project back up.
- A printable cheat sheet of the commands and idioms you'll use 90% of the time.

Prerequisites

- WSL2 Ubuntu 24.04 (or equivalent Linux) per [CLAUDE.md](#) global instructions.
 - VS Code with the **Remote - WSL** and **Dev Containers** extensions.
 - A working Docker engine in WSL (Docker Desktop with WSL integration, or `docker.io` installed in the distro).
 - CS-literate background. Familiarity with Python is assumed. Familiarity with the old PyData stack (pandas/jupyter/matplotlib) is helpful for the comparisons but not required.
 - ~30 minutes to set up the Dev Container the first time; ~3 minutes for every subsequent project.
-

Table of contents

1. [Mental model: what "scientific python" means in 2026](#)
 2. [uv](#): the universal tool
 3. [Project structure for data work](#)
 4. [DataFrames: Polars \(default\), Pandas \(when forced\)](#)
 5. [Notebooks: Marimo \(default\), Jupyter \(fallback\)](#)
 6. [Plotting: Altair, Matplotlib, Plotly, Seaborn](#)
 7. [Statistics and classical ML](#)
 8. [Deep learning, briefly](#)
 9. [Reproducibility](#)
 10. [From notebook to module: when and how](#)
 11. [A Dev Container template](#)
 12. [Common workflows](#)
 13. [Anti-patterns](#)
 14. [Alternatives considered](#)
 15. [Cheat sheet](#)
-

1. Mental model: what "scientific python" means in 2026

"Scientific Python" is the stack of libraries that grew out of NumPy in the early 2000s and ate the data-analysis world over the next two decades. The 2015-era version of it — NumPy + pandas + Jupyter + matplotlib + scikit-learn — is what most tutorials, course material, and Stack Overflow answers still reference. That stack still works. It's also showing its age in three specific places:

- **pandas is slow and memory-hungry.** Built for single-machine in-memory analysis from ~2008, with a column-and-index data model that fights you the moment your data shape is non-trivial. Polars (2020+) is 5-30× faster on most operations, uses less memory, and has a query optimizer.
- **Jupyter notebooks have a hidden-state problem.** Cells can be run out of order, deleted after defining variables, or re-run with different inputs. A notebook that "works" on your screen may not work for anyone else (including future-you) because the state on disk doesn't match the state in your kernel. Marimo (2023+) replaces the imperative cell model with a reactive dependency graph stored as plain `.py` — solving the reproducibility problem at the file format level.
- **pip + venv + pip-tools + pyenv is a Rube Goldberg machine.** Five tools each handling one slice of the "manage Python and its dependencies" problem. `uv` (2024+) is one binary, written in Rust, 10-100× faster than the tools it replaces, and ships a lockfile format the ecosystem is converging on.

The pieces that haven't changed are equally informative:

- **NumPy** is still the foundation. Every numeric library in the ecosystem speaks NumPy arrays at some interface boundary.
- **scikit-learn** is still the default for classical ML. Logistic regression, random forests, gradient boosting, cross-validation, pipelines, preprocessing — its API and breadth have no real

competitor. Some teams use XGBoost or LightGBM directly for tabular ML competitions, but they slot into the scikit-learn pipeline API.

- **SciPy** is still the standard library for stats, signal processing, optimization, sparse matrices. Boring, stable, indispensable.
- **statsmodels** is still where you go for inferential stats (OLS with proper confidence intervals, GLMs, time series, hypothesis tests). Scikit-learn intentionally doesn't do inference; statsmodels does.

So the 2026 stack is **mostly NumPy/SciPy/scikit-learn at the foundation** (unchanged), with the **dataframe/notebook/tooling layers replaced** by faster, less-foot-gun alternatives.

What replaced what

2015 stack	2026 default	What changed
<code>pip + venv + pip-tools</code> <code>+ pyenv</code>	<code>uv</code>	One Rust binary; 10-100× faster; built-in lockfile; manages Python itself.
<code>pandas</code>	<code>polars</code>	5-30× faster; lazy evaluation + query optimizer; cleaner API; better memory use.
<code>jupyter notebook</code> / <code>jupyterlab</code>	<code>marimo</code>	Reactive (no hidden state); stored as <code>.py</code> (git-friendly); built-in UI elements.
<code>matplotlib</code> (everything)	<code>altair</code> for declarative, <code>matplotlib</code> for control	Altair is grammar-of-graphics → cleaner; matplotlib still wins for fine control.
<code>conda</code> / <code>anaconda</code>	<code>uv</code> + system packages via apt or container	conda was needed to manage non-Python deps (CUDA, MKL); containers solve that better.
<code>requirements.txt</code>	<code>pyproject.toml</code> + <code>uv.lock</code>	Standardized in PEP 621; lockfile means reproducible installs.

Everything else (NumPy, SciPy, scikit-learn, statsmodels, PyTorch) is unchanged in role.

Why the stack matters for MSDS coursework

Coursework has a specific shape: you do an analysis, write it up, submit a PDF, and may or may not pick the project up again later. Three properties matter:

1. **The notebook you submitted has to match the notebook on your laptop six months later.** This is the reproducibility property. Hidden-state notebooks fail this often. Marimo or strict cell-order discipline pass it.
2. **You'll want to reuse code across assignments.** Tidy data-loading code, plotting helpers, model wrappers. The project structure has to make that easy without forcing premature abstraction.
3. **The PDF has to look good.** Quarto + matplotlib (with the `Charter`/`Georgia` font matched to the document) is the path of least resistance; Altair → PNG works too. Marimo can export to HTML/PDF directly.

The rest of this doc is calibrated for that shape: personal data projects and MSDS-style assignments, on a single laptop, with the occasional dataset large enough that Polars's memory efficiency starts to matter.

2. `uv`: the universal tool

IMPORTANT. `UV` REPLACES `PIP`, `VENV`, `PIP-TOOLS`, `PIPX`, `PYENV`, `POETRY`, AND `VIRTUALENV` — ALL OF THEM, WITH ONE BINARY. If you're still using any of those individually in 2026, stop. The switch is 10 minutes; the payoff is permanent.

Why `uv` exists

For two decades, Python's packaging story was a layer cake of partial solutions:

- `pip` installs packages, but doesn't isolate them.
- `venv` (or `virtualenv`) isolates them, but doesn't lock versions.
- `pip-tools` (`pip-compile` + `pip-sync`) locks versions, but is slow and externally maintained.
- `pipx` installs CLI tools in isolated envs (because installing Black globally polluted your Python).
- `pyenv` manages Python versions (because the OS ships an outdated Python).
- `poetry` tried to unify several of these, but its resolver is slow and it has its own opinions about project layout.

This was the state of the art until 2024, when Astral (the people who made `ruff`) released `uv`. Written in Rust, with a real SAT-based dependency resolver, it does **all of the above** — orders of magnitude faster than any of the tools it replaces. The result: `pip install` (~30 seconds) becomes `uv add` (~1 second). Resolving a complex dependency graph that took `pip-tools` 90 seconds takes `uv` under 2.

Install `uv`

```
# One command, works on Linux/macOS/WSL.
curl -LsSf https://astral.sh/uv/install.sh | sh

# Then make sure ~/.local/bin is on $PATH (the installer adds the export to ~/.bashrc).
# Verify:
uv --version
```

TIP. Inside a Dev Container, install `uv` in the Dockerfile via the official base image: `COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv`. That's a one-line, version-pinned install — no `curl` | `sh` needed inside the container.

The five commands you'll actually use

```
# Start a new project.
uv init my-analysis
cd my-analysis

# Add a dependency.
uv add polars altair marimo scikit-learn

# Add a dev-only dependency.
uv add --dev ruff pytest

# Sync the env to the lockfile (clean install).
uv sync

# Run a command inside the project's env.
uv run python analysis.py
uv run marimo edit notebook.py
```

That's the whole loop for 95% of work. `uv init` creates a `pyproject.toml` and `.python-version`; `uv add` writes the dep into `pyproject.toml`, resolves, installs, and updates `uv.lock`; `uv sync` reads the lockfile and reproduces the env exactly.

What `uv init` produces

```
my-analysis/
├── .python-version      # "3.12" – uv will install this Python if missing
├── README.md           # boilerplate
├── pyproject.toml       # PEP 621 project metadata + dependencies
└── main.py             # placeholder entrypoint (delete or repurpose)
```

The `pyproject.toml` looks like this:

```
[project]
name = "my-analysis"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []

[tool.uv]
dev-dependencies = []
```

After `uv add polars altair`:

```
[project]
# ...
dependencies = [
    "altair>=5.4.0",
    "polars>=1.5.0",
]
```

And a `uv.lock` file appears, containing the exact resolved versions of every dep and transitive dep. **Commit both files.** The `pyproject.toml` is the declaration; the `uv.lock` is the reproducible build.

Ephemeral envs via `uv run --with`

Sometimes you want to run a script that uses one or two libraries without creating a project at all. `uv run --with` does that:

```
# Run a script with polars available, in a throwaway env.
uv run --with polars python -c "import polars as pl; print(pl.__version__)"

# Run a Jupyter notebook with specific deps, no project setup.
uv run --with jupyterlab --with polars --with altair jupyter lab
```

The env is built in a cache, used, then garbage collected. No project, no venv, no cleanup. This replaces the entire use case for `pipx run` plus the "just install jupyter globally" hack.

Managing Python itself

`uv` installs and manages Python versions. No more `pyenv`.

```
# List available Python versions.
uv python list

# Install Python 3.13.
uv python install 3.13

# Pin this project to 3.13.
uv python pin 3.13 # writes .python-version

# Run with a specific Python without changing the project.
uv run --python 3.11 python --version
```

The `.python-version` file is the convention `pyenv` established; `uv` respects it. When you `cd` into a project with `.python-version = 3.12` and Python 3.12 isn't installed, `uv` will install it the first time you run anything.

Why not pip, poetry, or conda

Tool	Why not in 2026
<code>pip + venv</code>	Works, but slow, no lockfile, no Python management, no project metadata. <code>uv</code> does everything it does, faster and better-integrated.
<code>pip-tools</code>	The "lock with pip" workaround. <code>uv</code> has lockfiles natively and resolves 50× faster.
<code>poetry</code>	Was the best option pre- <code>uv</code> . Still works, but the resolver is slow, the lock format isn't standard, and <code>uv</code> is strictly faster and simpler. New projects: <code>uv</code> . Existing poetry projects: convert when you're touching them anyway (<code>uv</code> can import from <code>pyproject.toml</code>).
<code>pyenv</code>	Just manages Python versions. <code>uv</code> does that plus everything else.
<code>conda</code> / <code>anaconda</code>	Heavyweight (~3GB install), slow resolver, used to be necessary for CUDA/MKL but containers solve that better. Reconsider only if your org mandates it or you're on a Windows machine without WSL.
<code>pipx</code>	For installing CLI tools in isolated envs (Black, Ruff, etc.). <code>uv tool install</code> does the same thing. <code>uvx <command></code> is the equivalent of <code>pipx run</code> .

CAUTION. NEVER `PIP INSTALL` OUTSIDE A VENV ON THE HOST. It pollutes the system Python (or worse, breaks it on Ubuntu, where the system Python is used by `apt`). If you find yourself wanting to do this, the answer is `uv tool install <thing>` or `uv run --with <thing> <command>`.

3. Project structure for data work

A data project has a particular shape that web app projects don't share:

- **Raw data** (often large, often binary, often immutable once received).
- **Notebooks** for exploration and one-off analysis.
- **Scripts** for reusable, headless work (downloading data, running pipelines).
- **Modules** for code that's been promoted from notebook to reusable library.
- **Output** (plots, model artifacts, cleaned data, reports).

The structure below works for the spectrum from "one-off MSDS assignment" to "personal project that runs for months." It's deliberately simple — every extra directory is a future maintenance cost.

```

my-analysis/
├── .devcontainer/
│   ├── devcontainer.json
│   └── Dockerfile
├── .gitignore
├── .python-version
├── pyproject.toml
├── uv.lock
├── README.md
├── data/
│   ├── raw/                # immutable, gitignored if large
│   │   └── .gitkeep
│   ├── interim/           # intermediate cleaned data
│   └── processed/         # final analysis-ready data
├── notebooks/             # Marimo .py files (or .ipynb if fallback)
│   ├── 01-exploration.py
│   ├── 02-feature-engineering.py
│   └── 03-modeling.py
├── src/
│   ├── myproject/         # importable Python package
│   │   ├── __init__.py
│   │   ├── data.py        # data loading + cleaning
│   │   ├── features.py    # feature engineering
│   │   ├── models.py      # model training + eval
│   │   └── plots.py       # reusable plotting helpers
├── scripts/               # headless, runnable scripts
│   ├── download_data.py
│   └── train_model.py
└── output/
    ├── figures/           # plots, gitignored if large
    └── models/            # trained model artifacts, gitignored

```

Why each piece is the way it is

data/raw/ is sacred. Once data lands there, you never edit it. Cleaning produces new files in `interim/` or `processed/`. The principle is **idempotency**: running the analysis from scratch on raw data should produce the same results every time. If you mutate raw data, the analysis is no longer reproducible.

notebooks/ are numbered. `01-exploration.py`, `02-feature-engineering.py`. The numbers tell future-you (and any collaborator) the intended reading order. This is the cheap version of an analysis pipeline — formal pipelines (Snakemake, Prefect) are overkill for personal work.

src/myproject/ is the importable package. Code that started as cells in a notebook but is now reused goes here. The Marimo notebook `from myproject.features import standardize_columns` imports it back. See [section 10](#) for the workflow.

scripts/ is for headless runs. Anything you'd want to run from cron, from a CI job, or from "just rerun the whole pipeline" lives here. Scripts import from `src/myproject/`.

output/ is downstream. Everything in `output/` should be regenerable from the code + data. Don't put anything irreplaceable here.

`.gitignore` essentials

```
# Python
__pycache__/*
*.py[cod]
.venv/
.uv/
.ruff_cache/
.pytest_cache/

# Notebooks
.ipynb_checkpoints/
*.ipynb_convert/

# Data – gitignore large files; commit small ones with a note in README
data/raw/*
data/interim/*
data/processed/*
!data/raw/.gitkeep
!data/interim/.gitkeep
!data/processed/.gitkeep

# Output
output/figures/*
output/models/*
!output/figures/.gitkeep
!output/models/.gitkeep

# Secrets
.env
*.key
```

NOTE. FOR MSDS COURSEWORK, COMMIT SMALL DATASETS (<10 MB) DIRECTLY. The grader/future-you doesn't have access to your S3 bucket. For larger data, gitignore it and add a `data/raw/README.md` explaining the source URL and the download command.

When notebooks vs scripts vs both

Situation	Use
Exploring new data, plotting, trying things	Notebook
Final analysis you'd submit / share	Notebook (Marimo exports to HTML/PDF cleanly)
Code reused across notebooks	Module in <code>src/myproject/</code>
Long-running data download or model train	Script in <code>scripts/</code>
Something cron will run	Script in <code>scripts/</code>
Something with a Web UI / interactivity	Marimo notebook → <code>marimo run</code> (serves as app)

The general flow is: notebook for thinking → module when reused → script when headless.

4. DataFrames: Polars (default), Pandas (when forced)

Why Polars

Pandas was built in 2008, when "big data" meant 100 MB. Its data model — a column-oriented DataFrame indexed by a separate Index object — was a clever fit for the problems of that era but has aged badly:

- **It's slow.** Operations execute eagerly, one at a time, with no query optimization.
- **It's memory-hungry.** Internal representation often uses 2-3× the memory of the raw data.
- **The API is inconsistent.** `df.loc[]`, `df.iloc[]`, `df.at[]`, `df.iat[]`, `df.query()`, `df.eval()`, `df.apply()`, `df.transform()`, `df.agg()` — many do similar things with subtly different semantics.
- **The Index is a footgun.** Reset it, set it, MultiIndex it — many bugs hide in operations that silently change index alignment.
- **Strings are slow.** Until very recently, pandas stored strings as Python objects in a NumPy array. Polars stores them as Arrow.

Polars (2020+, by Ritchie Vink) was built from a different starting point:

- **Apache Arrow** as the in-memory representation. Columnar, zero-copy, language-portable.
- **Rust core**, with Python bindings.
- **Lazy evaluation with a query optimizer.** You describe what you want; Polars figures out the cheapest way to compute it.
- **Expression-based API.** Every operation is a composable expression — `pl.col("price") * pl.col("quantity")` — not a method on a Series.
- **No index.** Rows are just rows. Joins are explicit on key columns.
- **Multithreaded by default.** Polars uses all your cores for free.

The result: **5-30× faster than pandas** on typical workloads, **2-5× less memory**, and a consistent, composable API.

Performance comparison

These numbers are approximate, drawn from the [TPC-H benchmark](#) and consistent with my own experience on MSDS-scale datasets (1-100M rows):

Operation	Pandas (s)	Polars (s)	Speedup
Read 1M-row CSV	2.5	0.4	6×
<code>groupby().agg()</code> on 10M rows	4.2	0.3	14×
Join two 10M-row tables	8.1	0.5	16×
Filter + select on 100M rows (lazy)	22.0	0.8	27×

IMPORTANT. Polars's biggest single advantage isn't raw speed; it's **MEMORY EFFICIENCY ON MEDIUM-LARGE DATA**. Pandas operations that OOM your 16GB laptop often complete with room to spare in Polars.

API comparison: side-by-side

The thing that throws pandas users about Polars is the **expression API**. In pandas, you call methods on DataFrames and Series, mutating or returning new objects. In Polars, you build up expressions and pass them to `.select()`, `.filter()`, `.with_columns()`, `.group_by().agg()`.

```
# Read a CSV, filter, group, aggregate.

# — Pandas —————
import pandas as pd

df = pd.read_csv("sales.csv")
df = df[df["country"] == "US"]
result = df.groupby("product").agg(
    total_revenue=("revenue", "sum"),
    avg_price=("price", "mean"),
    n_orders=("order_id", "count"),
).reset_index().sort_values("total_revenue", ascending=False)

# — Polars (eager) —————
import polars as pl

df = pl.read_csv("sales.csv")
result = (
    df.filter(pl.col("country") == "US")
    .group_by("product")
    .agg(
        pl.col("revenue").sum().alias("total_revenue"),
        pl.col("price").mean().alias("avg_price"),
        pl.col("order_id").count().alias("n_orders"),
    )
    .sort("total_revenue", descending=True)
)

# — Polars (lazy – recommended for any non-trivial pipeline) –
result = (
    pl.scan_csv("sales.csv")                # scan, not read
    .filter(pl.col("country") == "US")
    .group_by("product")
    .agg(
        pl.col("revenue").sum().alias("total_revenue"),
        pl.col("price").mean().alias("avg_price"),
        pl.col("order_id").count().alias("n_orders"),
    )
    .sort("total_revenue", descending=True)
    .collect()                             # materialize at the end
)
```

The lazy version is what unlocks the query optimizer. Polars looks at the full chain, pushes the filter down before the read, only reads the columns it needs, and parallelizes the group_by — all without you doing anything.

A few more idioms worth knowing

```
# Read various formats.
df = pl.read_csv("file.csv")
df = pl.read_parquet("file.parquet")      # preferred binary format
df = pl.read_json("file.json")
df = pl.read_excel("file.xlsx")           # requires `fastexcel` extra

# Lazy reads (recommended).
lf = pl.scan_csv("big.csv")
lf = pl.scan_parquet("big.parquet")       # super fast over partitioned datasets

# Column operations.
df = df.with_columns(
    (pl.col("price") * pl.col("quantity")).alias("total"),
    pl.col("date").str.to_date("%Y-%m-%d"),
    pl.col("category").cast(pl.Categorical),
)

# Filtering with multiple conditions.
df = df.filter(
    (pl.col("country") == "US") & (pl.col("date") >= pl.date(2024, 1, 1))
)

# Joins.
df = df.join(other, on="user_id", how="left")  # how: inner/left/outer/cross/semi/anti

# Window / rolling.
df = df.with_columns(
    pl.col("revenue").rolling_mean(window_size=7).over("product").alias("revenue_7d")
)

# Convert to/from pandas/numpy when forced.
df_pd = df.to_pandas()
df_np = df.to_numpy()
df = pl.from_pandas(df_pd)
```

When to reconsider Pandas

Situation	Why pandas wins
Library you depend on returns only <code>pd.DataFrame</code>	scikit-learn (mostly), seaborn, statsmodels, many domain libs. Use <code>.to_pandas()</code> at the boundary.
You're maintaining a legacy codebase	Don't rewrite working pandas to Polars for its own sake.
Time series with index-heavy semantics	Pandas's <code>DatetimeIndex</code> , resampling, and <code>tz_convert</code> are very polished. Polars has reached parity for most cases but pandas is still slightly nicer for index-pivoted time series.
Your data is small (<100k rows) and you already know pandas	Polars's win on small data is real but small; ergonomic familiarity matters more.
Working in a course that requires pandas	Use what the course requires. Then convert when you can.

TIP. THE PRAGMATIC POLICY: WRITE POLARS BY DEFAULT. CONVERT TO PANDAS AT THE BOUNDARY WITH LIBRARIES THAT REQUIRE IT. Most boundary cases are `.to_pandas()` once at the end of a pipeline before handing off to scikit-learn or seaborn.

Failure mode: implicit type coercion

```
# Pandas: silently coerces numeric columns to float64 if any null exists.
>>> df = pd.DataFrame({"x": [1, 2, None]})
>>> df["x"].dtype
dtype('float64') # the integer column became float because None became NaN

# Polars: preserves the integer type, uses null instead of NaN.
>>> df = pl.DataFrame({"x": [1, 2, None]})
>>> df.schema
Schema([('x', Int64)])
>>> df["x"].to_list()
[1, 2, None]
```

This is a small thing that turns into a big thing when you're joining on a column and one side has been silently upcast to float while the other is int — the join silently produces no matches. Polars distinguishes nulls from NaN; pandas conflates them.

5. Notebooks: Marimo (default), Jupyter (fallback)

Why notebooks at all

Notebooks exist because data work is interactive. You load a dataset, look at it, plot it, try a model, plot the residuals, change a feature, replot. Compile-run-edit loops fight this. REPL is faster but loses the visualizations and the narrative. The notebook is the right shape: code + output + prose, interleaved, persistent.

Why Marimo over Jupyter

Jupyter (born 2014 as IPython Notebook) has one giant problem: **hidden state**. Cells can be:

- Executed out of order.
- Deleted after defining variables that other cells still use.
- Re-executed with stale dependencies.
- Skipped entirely while other cells still depend on them.

The result is the classic "notebook works on my screen but not when restarted from scratch." Academic work, code review, and production handoffs all suffer for it.

Marimo (2023+, by Akshay Agrawal et al.) solves this at the **file format level**:

- **Stored as** `.py`, not `.ipynb`. Diffs cleanly in git. No JSON noise, no embedded outputs.
- **Reactive**: edit a cell, all downstream dependents re-run automatically. Like a spreadsheet.
- **No hidden state**: the dependency graph is static. If a cell references `df` and you delete the cell that creates `df`, Marimo tells you.
- **Built-in UI elements**: sliders, dropdowns, text inputs that bind to Python variables — no `ipywidgets` setup.

- **Runs as an app:** `marimo run notebook.py` serves the notebook as a web app (read-only or interactive), good for sharing with collaborators or future-you.
- **Same file works as a script:** `python notebook.py` runs it top to bottom as a regular script.

Install and run

```
uv add marimo
uv run marimo edit notebook.py      # opens an editor at localhost:2718
uv run marimo new                   # blank new notebook
uv run marimo run notebook.py      # serve as read-only app
```

A Marimo notebook example

This is what a Marimo file actually looks like on disk — readable Python:

```

import marimo

__generated_with__ = "0.8.0"
app = marimo.App(width="medium")

@app.cell
def __():
    import polars as pl
    import altair as alt
    return alt, pl

@app.cell
def __(pl):
    df = pl.read_csv("data/raw/penguins.csv")
    df.head()
    return (df,)

@app.cell
def __(mo):
    species_picker = mo.ui.dropdown(
        options=["Adelie", "Gentoo", "Chinstrap"],
        value="Adelie",
        label="Species",
    )
    species_picker
    return (species_picker,)

@app.cell
def __(alt, df, pl, species_picker):
    filtered = df.filter(pl.col("species") == species_picker.value)
    chart = (
        alt.Chart(filtered.to_pandas())
        .mark_circle(size=60)
        .encode(
            x="bill_length_mm:Q",
            y="bill_depth_mm:Q",
            color="island:N",
        )
    )
    chart
    return chart, filtered

@app.cell
def __():
    import marimo as mo
    return (mo,)

if __name__ == "__main__":
    app.run()

```

The `return` lines at the end of each cell declare what the cell exports to the dependency graph. Marimo writes them automatically; you don't think about them.

Compare that to a `.ipynb` file (JSON, mixed with base64-encoded image outputs, ~50 lines per cell) and the reason Marimo files diff cleanly in git becomes obvious.

Reactivity in practice

In the example above, when you change the dropdown from "Adelie" to "Gentoo", the cells that depend on `species_picker` re-run automatically — including the `filtered` cell and the `chart` cell. Cells that don't depend on it (like the imports) don't re-run. The dependency graph is automatic and explicit.

IMPORTANT. Marimo enforces a **DAG OF CELL DEPENDENCIES** — you can't have cells that mutate shared state in non-obvious ways. If two cells both try to define `df`, Marimo will error. This is annoying the first time and a relief every subsequent time.

When to reconsider Jupyter

Situation	Why Jupyter wins
Your course / lab requires <code>.ipynb</code> submissions	You don't have a choice. Use Jupyter.
You need a specific Jupyter kernel extension	<code>nbgrader</code> (autograding), <code>RISE</code> (slideshows), some Spark/Dask extensions only target Jupyter.
You're using a hosted service that only supports Jupyter	Colab, Kaggle, Databricks notebooks — all Jupyter-flavored.
You're working on someone else's existing <code>.ipynb</code>	Don't rewrite working code to Marimo for its own sake.
Heavy use of <code>ipywidgets</code> -specific interactive plots	Some plotting libraries (<code> bqplot </code> , certain Plotly modes) integrate deeply with <code>ipywidgets</code> and don't transfer cleanly.

For new personal/MSDS work that doesn't have one of those constraints, **default to Marimo**. The git-diff-ability alone is worth it; the reactivity and no-hidden-state are bonus.

Jupyter when you must

```
uv add jupyterlab ipykernel
uv run jupyter lab
```

opens at localhost:8888

Tame the worst hidden-state behavior with:

- **Run ALL before committing.** Make sure the notebook works top-to-bottom.
- **Clear all outputs before committing** (Edit > Clear All Outputs). Otherwise every cell re-run produces noisy git diffs.
- `nbstripout` as a pre-commit hook auto-strips outputs.

```
uv add --dev nbstripout
uv run nbstripout --install
```

installs as a git filter

6. Plotting: Altair, Matplotlib, Plotly, Seaborn

Four libraries, four jobs. None of them is "the right one" for every plot. Pick the right one per plot, not per project.

The role of each

Library	Best for	Why
Altair	Declarative, "grammar of graphics" plots; exploratory analysis	Concise syntax; defaults look good; consistent API; renders to HTML interactively
Matplotlib	Final publication-quality plots; full control; PDFs	The lowest-level standard; every Python plotting lib renders to it eventually; Quarto/LaTeX-friendly
Plotly	Interactive plots embedded in web apps / dashboards	True interactivity (zoom, hover, callbacks); good for sharing analyses with non-technical viewers
Seaborn	Statistical defaults (pair plots, distribution plots, regression overlays)	Smart statistical defaults on top of matplotlib; less code for common patterns

Altair: the declarative default

Altair is the Python interface to [Vega-Lite](#) — a JSON-based declarative grammar of graphics from the UW Interactive Data Lab. You describe **what** you want plotted in terms of data + encodings + marks; Vega-Lite figures out the rest.

```
import altair as alt
import polars as pl

df = pl.read_csv("data/raw/penguins.csv").to_pandas() # altair takes pandas

chart = (
    alt.Chart(df)
        .mark_circle(size=60, opacity=0.7)
        .encode(
            x=alt.X("bill_length_mm:Q", title="Bill Length (mm)"),
            y=alt.Y("bill_depth_mm:Q", title="Bill Depth (mm)"),
            color=alt.Color("species:N", legend=alt.Legend(title="Species")),
            tooltip=["species", "island", "body_mass_g"],
        )
        .properties(width=600, height=400, title="Penguin bill dimensions by species")
        .interactive() # adds zoom + pan
)

chart.save("output/figures/penguins.html") # interactive HTML
chart.save("output/figures/penguins.png") # static PNG (needs vl-convert-python)
```

The grammar generalizes. Facet by category, add a regression line, change the mark from `circle` to `bar` — they're all small tweaks to the same structure, not "look up the bar chart tutorial."

TIP. Install `vl-convert-python` (`uv add vl-convert-python`) to save Altair charts as PNG/SVG/PDF. Without it, you can only save as HTML.

Matplotlib: the control case

Matplotlib (1.0 released 2003, by John Hunter) is the foundational plotting library. Every other library you'll use renders to it at some level. When you need precise control — exact axis ticks, complex layouts with shared axes, fine-grained text positioning, publication-quality PDF output — matplotlib is the answer.

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)

x = np.linspace(0, 10, 200)
axes[0].plot(x, np.sin(x), label="sin")
axes[0].set_title("Sine")
axes[0].set_xlabel("x")
axes[0].set_ylabel("y")
axes[0].legend()

axes[1].plot(x, np.cos(x), label="cos", color="orange")
axes[1].set_title("Cosine")
axes[1].set_xlabel("x")
axes[1].legend()

fig.suptitle("Trig functions")
fig.tight_layout()
fig.savefig("output/figures/trig.pdf", dpi=300, bbox_inches="tight")
```

For Quarto/LaTeX/printed PDFs, matplotlib + the right font (`Charter`, `Georgia`, or a serif of your choice via `plt.rcParams["font.family"]`) is the path of least resistance.

Seaborn: statistical defaults

Seaborn is a thin layer on matplotlib with smart statistical defaults. For pair plots, distribution comparisons, regression overlays, and quick categorical summaries, seaborn is one line of code where matplotlib would be ten.

```
import seaborn as sns

# A pair plot colored by species – three lines.
sns.set_theme(style="ticks")
penguins = sns.load_dataset("penguins")
sns.pairplot(penguins, hue="species")
```

Use seaborn for EDA quick wins; switch to matplotlib (or Altair) when you need a polished final plot.

Plotly: interactive for the web

Plotly produces interactive HTML plots — zoom, pan, hover, click. Use it when:

- You're embedding a plot in a Dash/Streamlit/Marimo app.
- You're sharing an HTML report and want the reader to explore.
- The interactivity itself is the point (e.g., a 3D scatter where rotation matters).

For static plots in a PDF, Altair or matplotlib is simpler.

```
import plotly.express as px

fig = px.scatter(
    df, x="bill_length_mm", y="bill_depth_mm",
    color="species", hover_data=["island", "body_mass_g"],
)
fig.write_html("output/figures/penguins-plotly.html")
```

Decision matrix

Need static publication PDF?	→ Matplotlib (or Altair → PNG)
Quick EDA, want sane defaults?	→ Altair or Seaborn
Statistical plot (pairplot, regplot)?	→ Seaborn
Need interactivity for web/app?	→ Plotly (or Altair's <code>.interactive()</code>)
Need fine control over layout?	→ Matplotlib
Embedding in a Marimo notebook?	→ Altair (renders natively)

7. Statistics and classical ML

scikit-learn: still the default

Nothing has replaced scikit-learn. It's been the standard for classical ML since 2007 and remains the best-organized, best-documented, most-tested library for:

- Linear/logistic regression, ridge, lasso, elastic net.
- Decision trees, random forests, gradient boosting.
- SVMs, k-NN, naive Bayes.
- K-means, DBSCAN, hierarchical clustering.
- PCA, t-SNE, UMAP (via `umap-learn`).
- Preprocessing (scaling, encoding, imputation).
- Pipelines + `ColumnTransformer`.
- Cross-validation, grid search, learning curves.
- Model evaluation metrics.

XGBoost, LightGBM, and CatBoost are faster gradient boosting implementations but slot into the scikit-learn `fit/predict` API and play nicely in pipelines.

The minimal classification example

```
import polars as pl
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

df = pl.read_csv("data/raw/penguins.csv").drop_nulls()

# Polars → pandas only at the sklearn boundary.
X = df.select(["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]).to_pandas()
y = df["species"].to_pandas()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)

# Cross-validate first to get a stable estimate.
cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print(f"CV accuracy: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}")

# Then fit on all training data and evaluate on held-out test.
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

That's the template for ~80% of classical ML work: split, cross-validate, fit, evaluate. Variations are the model class, the preprocessing pipeline, the search strategy, and the metric.

Pipelines: stop leaking test data into preprocessing

The most common scikit-learn footgun is preprocessing the full dataset (scaling, imputing, encoding) **before** splitting. This leaks test-set information into training and overstates model performance.

The fix: wrap everything in a `Pipeline`, which ensures preprocessing is fit only on training folds during cross-validation.

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

numeric_features = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm"]
categorical_features = ["island", "sex"]

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), numeric_features),
    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("encode", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical_features),
])

pipeline = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=1000)),
])

cv_scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring="accuracy")

```

Cross-validation now fits the imputer + scaler + encoder per fold on only the training portion. No leakage.

statsmodels: inference, not prediction

When you need confidence intervals, p-values, AIC/BIC, and the rest of the inferential machinery — use `statsmodels`. scikit-learn deliberately doesn't do this (it's a prediction-first library).

```

import statsmodels.api as sm

X = sm.add_constant(df.select(["bill_length_mm", "body_mass_g"]).to_pandas())
y = df["bill_depth_mm"].to_pandas()

model = sm.OLS(y, X).fit()
print(model.summary())    # full table: coefficients, std errors, t-stats, p-values, R², AIC

```

Use statsmodels when the question is "is this effect significant?" or "what's the confidence interval?" — not "how well does this predict?"

SciPy: the workhorse

`scipy.stats` has the distributions, hypothesis tests, and statistical tools that don't fit anywhere else.

```

from scipy import stats

# Two-sample t-test.
t_stat, p_value = stats.ttest_ind(group_a, group_b)

# Pearson correlation with p-value.
r, p = stats.pearsonr(x, y)

# Fit a distribution and check goodness of fit.
loc, scale = stats.norm.fit(data)
ks_stat, ks_p = stats.kstest(data, "norm", args=(loc, scale))

```

8. Deep learning, briefly

PyTorch: the default

PyTorch (2016+, Meta) is the deep learning framework of choice for research and increasingly for production. Define-by-run dynamic graphs, Pythonic API, broad ecosystem (`torchvision`, `torchaudio`, `transformers`, `lightning`, `accelerate`). For nearly every new project — academic or applied — PyTorch is the right pick.

```

import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_features, hidden, n_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden),
            nn.ReLU(),
            nn.Linear(hidden, n_classes),
        )

    def forward(self, x):
        return self.net(x)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = MLP(in_features=4, hidden=32, n_classes=3).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

# Standard train loop omitted; see PyTorch docs.

```

For non-trivial training loops, **PyTorch Lightning** (`uv add lightning`) eliminates boilerplate (loss tracking, checkpointing, multi-GPU) without hiding PyTorch from you.

JAX: when composable transforms matter

JAX is Google's research framework. NumPy-compatible API, but with composable function transforms: `jit` (compile), `grad` (auto-diff), `vmap` (vectorize), `pmap` (parallelize). For research code where you want to express a function and then differentiate, vectorize, and JIT-compile it in any combination — JAX is genuinely elegant.

Reconsider PyTorch in favor of JAX when:

- You're in the Google research ecosystem (Flax, Haiku, Optax).
- You're doing scientific computing that needs auto-diff but isn't standard deep learning (PINNs, optimization, MCMC).
- You want functional purity (PyTorch is OOP and stateful by design).

TensorFlow: when forced

TensorFlow 2.x is still maintained but has lost the ecosystem war for new projects. Use it when:

- You're maintaining a legacy TF codebase.
- You need TensorFlow Lite or TensorFlow.js for deployment (those toolchains have no PyTorch equivalent of equal maturity).
- A specific Google research artifact ships as TF.

Otherwise, default to PyTorch.

CUDA / GPU setup

IMPORTANT. GPU passthrough in WSL2 has its own footguns (NVIDIA driver on Windows, nvidia-container-toolkit in WSL, CUDA-enabled Docker base images). See [gpu-passthrough-for-wsl](#) for the full setup. The short version: don't install CUDA libraries directly in the WSL distro; use the official `nvidia/cuda` base image inside your Dev Container.

For CPU-only work (most coursework), `uv add torch` installs the CPU build, which is fine. For GPU work, the PyTorch installer URL specifies the CUDA version:

```
# CPU-only (default; fine for most MSDS work).
uv add torch torchvision

# GPU (CUDA 12.4); see pytorch.org for the current URL.
uv add torch torchvision --index-url https://download.pytorch.org/whl/cu124
```

The Dev Container is the right place to encode this — see [section 11](#).

9. Reproducibility

The whole reason for the careful tooling is **reproducibility**: that the analysis you ran today produces the same result six months from now, on a fresh machine, after a Python update.

Five pieces, in order of leverage

1. `uv.lock` — pins every dep + transitive dep to exact versions. Commit it. `uv sync` reproduces the env from it.
2. `.python-version` — pins the Python interpreter version. Commit it.
3. **Random seeds everywhere** — `random_state=42` on every `train_test_split`, every cross-validator, every model that supports it; `np.random.default_rng(seed)` for any NumPy randomness; `torch.manual_seed(seed)` for PyTorch.
4. **Data immutability** — `data/raw/` is never edited; downstream files are regenerable from it. Document the source URL in `data/raw/README.md`.

5. **Dev Container** — same OS, same system libs, same Python, same env. See [section 11](#).

Random seeds: a practical pattern

```
import os
import random
import numpy as np

SEED = 42

def set_global_seeds(seed: int = SEED) -> None:
    """Set every source of randomness we might touch."""
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    try:
        import torch
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
    except ImportError:
        pass

set_global_seeds()
```

Call this at the top of every notebook/script that involves stochasticity. **Then pass `random_state=SEED` explicitly to every scikit-learn function that accepts one.** Setting NumPy's global seed isn't enough — scikit-learn ignores it unless you pass it explicitly to the estimator/splitter.

Data versioning

For personal/MSDS work, **don't reach for DVC** (Data Version Control) unless you have a real reason. The overhead is non-trivial and the payoff requires multiple collaborators or genuinely changing datasets.

Instead:

- **Small datasets (<10 MB):** commit directly. Git handles it.
- **Medium datasets (10 MB - 1 GB):** gitignore the file, commit a script that downloads it (`scripts/download_data.py`), document the source in `data/raw/README.md`.
- **Large datasets (>1 GB):** as above, plus consider storing the file in Google Drive / S3 / your VPS for reliable re-download.

```
# scripts/download_data.py
"""Download raw data. Idempotent: skips files that already exist."""
from pathlib import Path
import urllib.request

DATA_DIR = Path(__file__).parent.parent / "data" / "raw"
DATA_DIR.mkdir(parents=True, exist_ok=True)

DATASETS = {
    "penguins.csv": "https://example.com/penguins.csv",
}

for filename, url in DATASETS.items():
    target = DATA_DIR / filename
    if target.exists():
        print(f"skip {filename} (exists)")
        continue
    print(f"downloading {filename}")
    urllib.request.urlretrieve(url, target)
```

When you reconsider DVC: a team with multiple people changing the same datasets, or datasets that genuinely evolve (model training corpora, image collections). Personal MSDS coursework rarely hits either.

10. From notebook to module: when and how

The pattern

A function that was a cell in `01-exploration.py` and is now used in `02-feature-engineering.py` and `03-modeling.py` should be **lifted into a module** in `src/myproject/`. The notebook then imports it back.

This is the single most useful refactoring move in data work. It:

- DRYs up the code (one definition, three uses).
- Makes the function testable.
- Makes the notebook shorter and more focused on the actual analysis.

The mechanics

Suppose you have this cell in `notebooks/01-exploration.py`:

```
@app.cell
def __(pl):
    def standardize_columns(df):
        """Convert columns to snake_case."""
        return df.rename({c: c.lower().replace(" ", "_") for c in df.columns})

    df = pl.read_csv("data/raw/sales.csv")
    df = standardize_columns(df)
    return df, standardize_columns
```

And you find yourself wanting to use `standardize_columns` in another notebook. Move it:

1. Create `src/myproject/data.py`:

```
# src/myproject/data.py
"""Data loading and cleaning utilities."""
import polars as pl

def standardize_columns(df: pl.DataFrame) -> pl.DataFrame:
    """Convert column names to snake_case."""
    return df.rename({c: c.lower().replace(" ", "_") for c in df.columns})

def load_sales(path: str = "data/raw/sales.csv") -> pl.DataFrame:
    """Load sales data with column names standardized."""
    return standardize_columns(pl.read_csv(path))
```

1. Make sure `src/myproject/__init__.py` exists (empty is fine).
2. Make sure `pyproject.toml` has the package configured so it's editable-installable:

```
[project]
name = "myproject"
version = "0.1.0"
requires-python = ">=3.12"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/myproject"]
```

1. `uv sync` will pick up the new layout and install `myproject` in editable mode.
2. In the notebook, replace the cell:

```
@app.cell
def __():
    from myproject.data import load_sales
    df = load_sales()
    return df, load_sales
```

Now any edit to `src/myproject/data.py` is immediately reflected in any notebook that imports from it. Edit the function, re-run the importing cell, and your notebook uses the new version.

TIP. In Jupyter, use the `%load_ext autoreload` + `%autoreload 2` magic at the top of the notebook so module changes are picked up without a kernel restart. Marimo does this automatically — module changes trigger reactive re-runs.

When not to lift

Don't lift one-off code into modules prematurely. The notebook is the right place for code that's:

- Specific to one analysis.
- Likely to be deleted next week.

- Tied to the narrative of the notebook (data quality checks, sanity prints).

Lift when **you've copied a function across two or more notebooks**, or **a function has stabilized enough to deserve a docstring**.

11. A Dev Container template

The whole stack belongs inside a Dev Container so it's reproducible, disposable, and consistent across machines. Here's the template I use for new data projects.

`.devcontainer/devcontainer.json`

```
{
  "name": "scientific-python-2026",
  "build": {
    "context": "..",
    "dockerfile": "Dockerfile"
  },

  // Mount the project into the container.
  "workspaceFolder": "/workspace",
  "workspaceMount": "source=${localWorkspaceFolder},target=/workspace,type=bind,consistency=cached",

  // Run as non-root user inside the container.
  "remoteUser": "vscode",

  // Forward Marimo and Jupyter ports.
  "forwardPorts": [2718, 8888],

  "customizations": {
    "vscode": {
      "extensions": [
        "ms-python.python",
        "ms-python.vscode-pylance",
        "ms-toolsai.jupyter",
        "charliermarsh.ruff",
        "tamasfe.even-better-toml",
        "redhat.vscode-yaml",
        "marimo-team.vscode-marimo"
      ],
      "settings": {
        "python.defaultInterpreterPath": "/workspace/.venv/bin/python",
        "python.terminal.activateEnvironment": false,
        "[python]": {
          "editor.formatOnSave": true,
          "editor.defaultFormatter": "charliermarsh.ruff",
          "editor.codeActionsOnSave": {
            "source.organizeImports.ruff": "explicit"
          }
        }
      }
    }
  },

  // Sync the env to the lockfile after the container is built.
  "postCreateCommand": "uv sync",

  // For GPU work, add the nvidia runtime – see gpu-passthrough-for-wsl.
  // "runArgs": ["--gpu=all"]
}
```

`.devcontainer/Dockerfile`

```
# syntax=docker/dockerfile:1.7

FROM python:3.12-slim

# Install system dependencies that scientific libs occasionally need.
# graphviz: for some scikit-learn / pydotplus model viz
# libpq-dev: for postgres clients
# git, curl, build-essential: general dev needs
RUN apt-get update && apt-get install -y --no-install-recommends \
    git \
    curl \
    ca-certificates \
    build-essential \
    graphviz \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Install uv from the official image – version-pinned, reproducible.
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

# Create a non-root user that matches the host UID/GID.
ARG USERNAME=vscode
ARG USER_UID=1000
ARG USER_GID=$USER_UID
RUN groupadd --gid $USER_GID $USERNAME \
    && useradd --uid $USER_UID --gid $USER_GID -m $USERNAME

ENV UV_LINK_MODE=copy \
    UV_COMPILE_BYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

WORKDIR /workspace
USER $USERNAME
```

What each piece does

- `python:3.12-slim` as the base. Small image (~50 MB), modern Python. The Dockerfile is the single source of truth for the Python version — even more authoritative than `.python-version`.
- `graphviz` + `libpq-dev` are the apt packages I've most often needed in scientific work. Add others as you hit them; this isn't an exhaustive list.
- `COPY --from=ghcr.io/astral-sh/uv:latest /uv` pulls the `uv` binary from the official image. Faster than `curl | sh` and version-pinnable (replace `:latest` with `:0.4.0` for reproducibility).
- **Non-root `vscode` user.** Important: when VS Code Dev Containers mounts your host workspace, files created in the container will be owned by whatever user the container runs as. A root container creates root-owned files on the host, which is annoying. The `vscode` user with UID 1000 (the typical default WSL user UID) keeps file ownership clean.
- **postCreateCommand: `uv sync`** runs once after the container is built. It reads `uv.lock` and creates `.venv/`. Subsequent rebuilds will reuse the cached `.venv` because it lives in the workspace mount.

Recommended VS Code extensions

Extension	Why
<code>ms-python.python</code>	Core Python support (debugging, env detection).
<code>ms-python.vscode-pylance</code>	Type checking + autocomplete. Significantly better than the default IntelliSense.
<code>ms-toolsai.jupyter</code>	Jupyter notebook editing in VS Code.
<code>charliermarsh.ruff</code>	Lint + format, one binary, very fast. Replaces flake8/isort/black.
<code>tamasfe.even-better-toml</code>	Syntax highlighting and validation for <code>pyproject.toml</code> and <code>uv.lock</code> .
<code>redhat.vscode-yaml</code>	YAML validation (for any config files).
<code>marimo-team.vscode-marimo</code>	First-class Marimo support inside VS Code.

Opening it

1. `mkdir my-analysis && cd my-analysis`
2. `uv init` (creates `pyproject.toml`, `.python-version`).
3. Drop the `.devcontainer/` files in.
4. `code .` (opens VS Code in WSL).
5. Command Palette → **"Dev Containers: Reopen in Container"**.
6. Wait ~60 seconds for the first build; subsequent opens are near-instant.

You're now in a Python 3.12 + uv environment, identical across every machine you open this repo on.

12. Common workflows

Starting a new analysis

```
# 1. Create the project.
mkdir ~/dev/my-analysis && cd ~/dev/my-analysis
uv init

# 2. Add the standard deps.
uv add polars altair marimo scikit-learn numpy scipy
uv add --dev ruff pytest

# 3. Drop in the Dev Container template (see section 11).
mkdir -p .devcontainer
# (paste devcontainer.json + Dockerfile)

# 4. Create the project structure.
mkdir -p data/{raw,interim,processed} notebooks scripts src/myproject output/{figures,models}
touch data/raw/.gitkeep notebooks/.gitkeep
touch src/myproject/__init__.py

# 5. Git init and first commit.
git init
git add .
git commit -m "initial scaffold"

# 6. Open in VS Code, reopen in container.
code .
```

About 2 minutes from "I have an idea" to "I have a working env, version-pinned and containerized."

Sharing a notebook with a collaborator (or future-you)

The hardest reproducibility problem isn't "the code works" — it's "the code works for someone else, on a different machine, six months later."

The recipe:

1. **Commit** `pyproject.toml` + `uv.lock` + `.python-version` + `.devcontainer/`. That's the env, exactly.
2. **Commit the notebooks as** `.py` (Marimo) **or** `.ipynb` **with outputs stripped.**
3. **Either commit the data** (small) **or commit a download script** (large), with the source URL documented.
4. **Set seeds at the top of every notebook.**
5. **Add a** `README.md` **with one section:** "How to run this":

```
## How to run

1. Open this repo in VS Code with the Dev Containers extension.
2. Reopen in Container.
3. `uv run python scripts/download_data.py`
4. `uv run marimo edit notebooks/01-exploration.py`
```

Test the recipe by **cloning the repo into a fresh directory and following it yourself**. If `uv sync` + `marimo edit` produces a working notebook, you're done.

Packaging up a finding as a reproducible artifact

When an analysis is done and you want it as a permanent record (course submission, blog post, internal report):

1. **Run the whole pipeline from scratch** — delete `data/processed/`, `output/`, restart kernels, re-run everything. If it doesn't reproduce, fix it before you publish.
2. **Marimo → HTML**: `uv run marimo export html notebooks/03-modeling.py -o output/report.html`. Self-contained HTML, no kernel required.
3. **Marimo → PDF**: print the HTML, or use Quarto if you want LaTeX-quality typesetting.
4. **Tag the commit**: `git tag analysis-v1` and push. The lockfile + tag is your permanent reproducibility handle.

13. Anti-patterns

These show up in scientific Python work constantly. Each one has a known better path; the fix is short.

`pip install` outside a venv

Symptom: You run `pip install pandas` and either it pollutes your system Python or (on Ubuntu 23.04+) it fails with "externally-managed-environment."

Why it's bad: System Python is used by `apt` and the OS. Polluting it breaks the OS. Even in a venv, raw `pip install` doesn't update a lockfile.

Fix: `uv add <pkg>` inside a project, or `uv run --with <pkg> <command>` for ephemeral use.

`from foo import *` in notebooks

Symptom: Notebook has `from numpy import *` or `from pandas import *` at the top.

Why it's bad: Pollutes the namespace; obscures where names come from; can shadow built-ins (`import * from numpy` shadows `min`, `max`, `sum`, `any`, `all`).

Fix: Always `import polars as pl` / `import numpy as np`. Two extra characters per use; massive clarity win.

Mixing notebook state with script imports

Symptom: A script imports a notebook (via `nbimporter` or similar), or a notebook does `exec(open("other_notebook.py").read())`.

Why it's bad: Couples two artifacts that are supposed to be independent. Hidden state proliferates. Breaks when someone re-runs the notebook in a different order.

Fix: Lift shared code into `src/myproject/`. The notebook and the script both import from there.

Plotting in matplotlib then trying to embed in a web app

Symptom: You built a matplotlib plot for a report, then want to put it in a Streamlit/Dash/Marimo app and discover it doesn't interact.

Why it's bad: Matplotlib is for static output. Re-rendering it on a web page works but interactivity (zoom, hover) does not.

Fix: Pick the plotting library based on the **destination**, not the workflow. For interactive web embedding, use Altair or Plotly from the start.

Credentials in notebooks

Symptom: A notebook has `API_KEY = "sk-abc123..."` somewhere in a cell. Then you `git push`.

Why it's bad: Notebooks (especially `.ipynb`) serialize outputs and code. The secret is in the file forever; even amending the commit doesn't help if you've already pushed. Even Marimo's `.py` notebooks are easy to leak from.

Fix: Use environment variables. `.env` in the project root (gitignored); read via `os.environ` or `python-dotenv`. The notebook reads the var; the secret stays out of git.

```
import os
from dotenv import load_dotenv

load_dotenv()
api_key = os.environ["OPENAI_API_KEY"]
```

CAUTION. If you've ever committed a credential, **ROTATE IT**. Even if you force-push to remove it from history, assume it's compromised. Git history is forever for anything that's been pushed.

Mutating `data/raw/`

Symptom: "I'll just clean this column in place to save time."

Why it's bad: Now the raw data is no longer the raw data. The next person (including future-you) running the analysis from scratch gets a different starting point.

Fix: Read from `data/raw/`, write cleaned versions to `data/interim/` or `data/processed/`. Never write to `raw/`.

Cross-validating after preprocessing

Symptom: You scale the whole dataset, then split into train/test, then cross-validate.

Why it's bad: Test-set statistics leaked into the scaler. Reported performance is optimistic. Common in beginner tutorials and beginner-graded assignments.

Fix: Wrap preprocessing in a `Pipeline`. The pipeline is what you pass to `cross_val_score` or `GridSearchCV`. Preprocessing is then fit per-fold on only the training portion.

Trusting accuracy on imbalanced data

Symptom: "My model has 99% accuracy!" on a dataset where 99% of samples are class 0.

Why it's bad: A model that always predicts class 0 has 99% accuracy and zero predictive value.

Fix: Use precision/recall/F1, ROC-AUC, PR-AUC. Plot the confusion matrix. Stratify your splits (`stratify=y` in `train_test_split`). For severe imbalance, consider class weights or resampling.

14. Alternatives considered

Anaconda / conda

Verdict: No, in 2026.

Conda was indispensable from ~2012 to ~2020 because it could install non-Python deps (CUDA, MKL, GDAL) that pip couldn't. That's no longer a meaningful advantage:

- Containers handle non-Python deps cleanly via `apt-get install`.
- `uv` installs Python itself.
- PyTorch ships CUDA-bundled wheels via `pip` for major platforms.

Conda is also slow, heavy (~3 GB install), and has its own incompatibility footguns (the `defaults` vs `conda-forge` channel mess).

Reconsider when: your org mandates conda; you're on a managed cluster that uses conda environments natively; you need a niche binary that's still conda-only (rare).

Pixi

`Pixi` is a newer (~2023) cross-language environment manager from Prefix.dev, built on the conda ecosystem but with `uv`-like ergonomics. It can install Python and R and Rust toolchains in the same env via conda-forge.

Verdict: Watch, but not yet for new Python-only projects. `uv` is faster and more focused for Python work.

Reconsider when: you have a project that genuinely needs multiple language toolchains together (Python + R for stats class, Python + CUDA + custom C++); or when pixi reaches the maturity/adoption that uv has now.

Poetry

The previous-generation "modern" Python tool (2018+). Was strictly better than pip+venv+pip-tools and dominated 2020-2024.

Verdict: Use `uv` for new projects. Convert existing Poetry projects to `uv` when you're touching them anyway — `uv` reads Poetry's `pyproject.toml` format and migration is largely automatic (`uv lock` from a Poetry pyproject works).

Reconsider when: the project you're contributing to uses Poetry and switching isn't your call.

virtualenv + pip + requirements.txt

The 2010-era stack.

Verdict: No. Slow, no lockfile, no Python management, no integrated tooling. Every layer is solved better by `uv`.

Reconsider when: you're on a locked-down corporate machine where `uv` can't be installed. Even then, fight for it.

Dask / Ray / Spark / Modin

Distributed dataframe / compute libraries. Used to be the answer for "data bigger than memory" before Polars's lazy mode existed.

Verdict: Don't reach for these until Polars genuinely can't handle your data. For MSDS-scale work (up to ~100M rows, ~50 GB raw), a single machine running Polars on a laptop is fast enough and dramatically simpler.

Reconsider when: working with data that genuinely doesn't fit on one machine; on a cluster where the distributed scheduler is part of the assumed setup.

Notebook alternatives (Observable, Hex, Deepnote, Quarto-only)

Verdict: Marimo for the local interactive case; Quarto for the "publish as report" case; these others for hosted/team scenarios.

Reconsider when: you're working in a team that has standardized on one of them (Hex, Deepnote especially common in industry data teams); or you specifically want JavaScript-flavored notebooks (Observable).

MLflow / Weights & Biases / DVC for experiment tracking

For personal/MSDS work, **none of these are required**. A `results.csv` logged from your training script is fine. A git tag is fine.

Reconsider when: you're running 50+ experiments per project; you're collaborating with someone else and need to share artifacts; you're putting models into production.

15. Cheat sheet

This is the page to print and pin above the desk.

`uv` commands

Command	What it does
<code>uv init <name></code>	Create a new project.
<code>uv add <pkg></code>	Add a dep; update lockfile; install.
<code>uv add --dev <pkg></code>	Add a dev-only dep.
<code>uv remove <pkg></code>	Remove a dep.
<code>uv sync</code>	Reproduce env exactly from <code>uv.lock</code> .
<code>uv lock</code>	Update lockfile without installing.
<code>uv run <command></code>	Run a command inside the project env.
<code>uv run --with <pkg> <cmd></code>	Run a command with an ephemeral extra dep.
<code>uv python install <ver></code>	Install a Python version.
<code>uv python pin <ver></code>	Pin this project to a Python version.
<code>uv tool install <pkg></code>	Install a CLI tool globally (in its own env).
<code>uvx <cmd></code>	Run a CLI tool ephemeraly (= <code>pipx run</code>).
<code>uv cache clean</code>	Clean the package cache.

Polars common idioms

```
import polars as pl

# Read.
df = pl.read_csv("file.csv")
lf = pl.scan_csv("big.csv")           # lazy; recommended for large

# Inspect.
df.head(); df.tail(); df.schema; df.shape; df.describe(); df.null_count()

# Select / project.
df.select("col1", "col2")
df.select(pl.col("*").exclude("ignore_me"))

# Filter.
df.filter(pl.col("country") == "US")
df.filter((pl.col("x") > 0) & (pl.col("y").is_not_null()))

# Add / transform columns.
df.with_columns(
    (pl.col("price") * pl.col("qty")).alias("total"),
    pl.col("date").str.to_date("%Y-%m-%d"),
    pl.col("name").str.to_lowercase(),
)

# Group + aggregate.
df.group_by("category").agg(
    pl.col("price").mean().alias("avg_price"),
    pl.col("id").count().alias("n"),
)

# Join.
df.join(other, on="key", how="left")   # how: inner / left / outer / cross / semi /
anti

# Sort.
df.sort("col", descending=True)

# Window over a group.
df.with_columns(pl.col("revenue").sum().over("category").alias("cat_total"))

# Convert.
df.to_pandas(); df.to_numpy(); pl.from_pandas(df_pd)

# Save.
df.write_csv("out.csv"); df.write_parquet("out.parquet")
```

Marimo basics

```
uv add marimo
uv run marimo new           # new blank notebook
uv run marimo edit nb.py   # edit existing
uv run marimo run nb.py    # serve as read-only app
uv run marimo export html nb.py -o report.html
```

```
# Inside a Marimo notebook:
import marimo as mo

# UI elements bind to Python vars and trigger reactive re-runs.
slider = mo.ui.slider(0, 100, value=50, label="threshold")
dropdown = mo.ui.dropdown(["A", "B", "C"], value="A")
text = mo.ui.text(value="hello")

# Display markdown, dataframes, plots inline.
mo.md("## Heading + bold markdown")
mo.ui.table(df) # interactive dataframe view
```

Altair quick recipes

```
import altair as alt

# Scatter.
alt.Chart(df).mark_circle().encode(x="x:Q", y="y:Q", color="cat:N").interactive()

# Bar.
alt.Chart(df).mark_bar().encode(x="cat:N", y="count():Q")

# Line.
alt.Chart(df).mark_line().encode(x="date:T", y="value:Q", color="series:N")

# Histogram.
alt.Chart(df).mark_bar().encode(alt.X("x:Q", bin=True), y="count():Q")

# Facet (small multiples).
alt.Chart(df).mark_circle().encode(x="x:Q", y="y:Q").facet(column="category:N")

# Layer two marks.
(alt.Chart(df).mark_circle().encode(x="x:Q", y="y:Q")
 + alt.Chart(df).mark_line().encode(x="x:Q", y="y_smooth:Q"))

# Save.
chart.save("out.png") # needs vl-convert-python
chart.save("out.html")
```

Encoding type suffixes: `:Q` quantitative, `:N` nominal, `:O` ordinal, `:T` temporal.

scikit-learn train/score template

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

SEED = 42
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", RandomForestClassifier(n_estimators=200, random_state=SEED, n_jobs=-1)),
])

cv = cross_val_score(pipe, X_train, y_train, cv=5, scoring="accuracy")
print(f"CV: {cv.mean():.3f} ± {cv.std():.3f}")

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Pandas → Polars rosetta

Pandas	Polars
<code>pd.read_csv(p)</code>	<code>pl.read_csv(p)</code>
<code>df[df["x"] > 0]</code>	<code>df.filter(pl.col("x") > 0)</code>
<code>df[["a", "b"]]</code>	<code>df.select("a", "b")</code>
<code>df["c"] = df["a"] + df["b"]</code>	<code>df.with_columns((pl.col("a") + pl.col("b")).alias("c"))</code>
<code>df.groupby("k").agg({"v": "sum"})</code>	<code>df.group_by("k").agg(pl.col("v").sum())</code>
<code>df.merge(other, on="k", how="left")</code>	<code>df.join(other, on="k", how="left")</code>
<code>df.sort_values("c", ascending=False)</code>	<code>df.sort("c", descending=True)</code>
<code>df.head()</code>	<code>df.head()</code>
<code>df["x"].isna()</code>	<code>pl.col("x").is_null()</code>
<code>df["x"].fillna(0)</code>	<code>pl.col("x").fill_null(0)</code>
<code>df.rename(columns={"a": "b"})</code>	<code>df.rename({"a": "b"})</code>
<code>df.drop_duplicates()</code>	<code>df.unique()</code>
<code>df.to_numpy()</code>	<code>df.to_numpy()</code>
<code>df.to_dict()</code>	<code>df.to_dicts()</code> (list of row dicts)

IPython / Jupyter magics worth knowing

Magic	What it does
<code>%timeit <expr></code>	Time a single expression (multiple runs, statistical).
<code>%%timeit</code>	Time the whole cell.
<code>%time <expr></code>	Time once (no stats).
<code>%load_ext autoreload</code> + <code>%autoreload 2</code>	Auto-reload edited modules without kernel restart.
<code>%matplotlib inline</code>	Embed matplotlib plots in cells.
<code>%pdb on</code>	Drop into pdb on exceptions.
<code>?<obj></code>	Show object's docstring.
<code>??<obj></code>	Show object's source.
<code>!<cmd></code>	Run a shell command (e.g., <code>!ls data/raw/</code>).
<code>%env VAR=value</code>	Set an environment variable for the session.

Random seeds checklist

```
SEED = 42

# Python
import random; random.seed(SEED)

# NumPy (legacy global)
import numpy as np; np.random.seed(SEED)
# NumPy (modern; preferred)
rng = np.random.default_rng(SEED)

# Hash randomization
import os; os.environ["PYTHONHASHSEED"] = str(SEED)

# scikit-learn – pass to every function that accepts it
train_test_split(..., random_state=SEED)
KFold(..., random_state=SEED)
RandomForestClassifier(..., random_state=SEED)

# PyTorch
import torch
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
```

Dev container quickstart

```
# In a new project dir:
uv init
mkdir -p .devcontainer
# (paste devcontainer.json + Dockerfile from section 11)
mkdir -p data/{raw,interim,processed} notebooks scripts src/myproject output/{figures,models}
touch src/myproject/__init__.py
git init && git add . && git commit -m "scaffold"
code .                                # then: Dev Containers: Reopen in Container
```

Related how-tos

- [gpu-passthrough-for-wsl](#) — GPU-enabled Dev Containers for PyTorch/JAX work.
- [dockerized-deployments](#) — moving a Python project from Dev Container to production VPS.
- [vps-from-zero](#) — the VPS that hosts the production version.
- [git-for-solo-devs](#) — the git workflow that this project structure assumes.
- [claude-code-workflow](#) — using Claude Code productively on data work.